Grad-CAM: Visual Explanations for CNN Classifiers

Pattern Recognition (EE798R) — Course Project

Aaradhya Rohi (Roll no. 230011)

October 11, 2026

Abstract

This report studies the Grad-CAM method of Selvaraju et al. [16]. Deep convolutional networks classify images well but do not say *where* in the image the evidence for a decision lies. Gradient-weighted Class Activation Mapping (Grad-CAM) [16] answers this for almost any CNN by weighting the feature maps of a chosen convolutional layer with the spatially averaged gradient of the class score and keeping only positive evidence. This report derives the method from first principles, shows that it reduces to Class Activation Mapping (CAM) [25] on a global-average-pooling classifier, and re-implements it from scratch in PyTorch. We fine-tune an ImageNet-pretrained ResNet-50 on PASCAL VOC 2007 and measure how well the explanations localise objects, using ground-truth bounding boxes, against vanilla gradient saliency, CAM, Grad-CAM++ and a model-free centre prior. We further study sensitivity to the target layer and to the binarisation threshold, the class-discriminativeness of the maps, runtime, and failure cases. On the 7,013 image–class pairs of the VOC 2007 test set, Grad-CAM reaches 85.6% pointing accuracy and 38.8% box-localisation accuracy (IoU ≥ 0.5 at a threshold of 15% of the map maximum), against 63.1% and 23.1% for a model-free centre prior; its pointing accuracy is statistically identical to CAM and Grad-CAM++, and its box advantage over them at the default threshold largely disappears when each method uses its own best threshold. Deeper layers localise much better than shallow ones, Grad-CAM at the last stage costs about 2% over a forward pass, and the main failures are small and very large objects, part-only responses, context cues and low classifier confidence.

1 Introduction and Motivation

A classifier that reports "dog, 0.97" is hard to trust or debug unless we can tell what it looked at. Explanations are useful for (i) detecting shortcut learning, where the network relies on background or context instead of the object; (ii) understanding failures and misclassifications; (iii) building trust in safety-critical settings such as medical imaging; and (iv) scientific inspection of what a representation has learned. The difficulty is that a CNN is a composition of millions of parameters, so a faithful, human-readable summary of "where the evidence is" must be computed, not read off.

Early gradient-based visualisations are sharp but noisy and tied to single pixels, while CAM is coarse but class-discriminative and requires a special architecture. Grad-CAM was proposed to keep CAM's class-discriminative, region-level maps while removing the architectural restriction. In this report we study Grad-CAM as a *pattern-visualisation* tool and ask a concrete, measurable question: **when a CNN classifies a VOC image, how well does the Grad-CAM heat-map for a present class overlap the annotated object, and how does this compare with simpler explanation methods?**

Scope and contributions. (1) A self-contained derivation and interpretation of Grad-CAM (Sections 3–4), including a proof that CAM is a special case. (2) An independent PyTorch implementation of Grad-CAM, vanilla saliency and CAM, with unit tests that check the CAM identity numerically (Section 6). (3) A quantitative localisation study on the full VOC 2007 test set with confidence intervals, a threshold sweep, a layer ablation, a class-discrimination measure

and timings (Sections 7–8). (4) A qualitative study with rule-based (not hand-picked) examples of success, partial, background-dependent, misclassification and multi-object cases (Section 9). We do *not* train anything at scale, and we do not claim to reproduce the ImageNet experiments of the original paper.

How results are labelled. Three kinds of statements appear: numbers *reported by the original paper* (always marked as such and cited), numbers *measured by us* (generated by the released scripts and read into the report from files), and *expectations, hypotheses and interpretation* (explicitly worded as such, e.g. "hypothesis" or "plausible reading").

2 Background and Related Work

2.1 Deep Neural Network Interpretability

Interpretability methods for vision models fall into three families: *backpropagation-based* methods that use gradients through the network (saliency, deconvolution, guided backpropagation, CAM-family methods), *perturbation-based* methods that observe how the output changes when the input is altered (occlusion, LIME), and *representation-space* visualisations that embed internal features in a low-dimensional space (t-SNE [21], UMAP [10], Isomap [20], LLE [14], MDS [8], Mapper [18]). The last family explains the *geometry of a feature space* over many samples, not *which pixels of one image* drove one prediction; it is therefore outside the scope of Grad-CAM and of this project (see Table 2).

2.2 Saliency Maps

Idea: the pixels whose small change most affects the class score are important. **Mechanism**: Simonyan et al. [17] take $|\partial y^c/\partial \boldsymbol{x}|$ (maximum over colour channels) for input $\boldsymbol{x}$ and class score y^c. **Strength**: one backward pass, any differentiable model, pixel resolution. **Limitation**: the gradient is a *local* linearisation of a highly non-linear function, so maps are noisy, scattered and do not form object-shaped regions.

2.3 Deconvolution and Guided Backpropagation

Zeiler and Fergus [23] and Springenberg et al. [19] also propagate a signal back to the image but modify the rule at ReLU units (pass only positive signals back; guided backpropagation additionally requires the forward activation to be positive). **Strength**: visually sharp, edge-like detail. **Limitation**: the result is largely dependent on the input image rather than on the chosen class, and Adebayo et al. [1] report that some such methods are insensitive to the trained parameters, i.e. they behave partly like edge detectors. Grad-CAM was explicitly designed to supply the class-discriminative component these maps lack (the paper combines the two as Guided Grad-CAM; we do not study that combination).

2.4 Class Activation Mapping

Idea: if the last convolutional features are followed by global average pooling (GAP) and a single linear layer, then the class score is a weighted sum of spatially pooled feature maps, so the *un*pooled weighted sum is a spatial evidence map. **Mechanism**: $M^c_{ij} = \sum_k w^c_k A^k_{ij}$ with w^c_k the classifier weights [25]. **Strength**: class-discriminative, region-level, no gradients needed. **Limitation**: requires the GAP→linear architecture; networks such as VGG-16 with fully connected layers must be modified and retrained.

Table 1: CAM versus Grad-CAM. The last row is derived in Eq. (4) and verified numerically in our tests; the other rows follow [16, 25].

Property	CAM	Grad-CAM
Requires specific architecture?	Yes: last conv. layer → GAP → one linear layer	No: any CNN whose class score is differentiable w.r.t. a conv. layer
Requires GAP?	Yes	No
Requires classifier retraining/modification?	Yes, for networks not already of this form (e.g. replace FC layers by GAP + linear, retrain)	No: applied to the trained model as is
Uses gradients?	No: uses the classifier weights w_k^c	Yes: weights α_k^c from gradients
Applicable to arbitrary CNN architectures?	No	Yes (also to non-classification heads, per the paper)
Spatial localisation	Coarse: resolution of the last conv. map	Coarse: resolution of the chosen layer (shallower layers give a finer grid)
Class-specific explanation?	Yes	Yes
Relation	–	= CAM up to a positive constant on a CAM-type network

2.5 Grad-CAM

Idea: replace CAM's classifier weights by gradient-derived channel importances. **Mechanism**: Eqs. (3)–(5) below. **Strength**: works for any CNN whose class score is differentiable with respect to a convolutional layer, with no retraining; the original paper also applies it to captioning, visual question answering and reinforcement-learning agents [16]. **Limitation**: coarse (resolution of the target layer), and the highlighted region is not guaranteed to be a causal explanation (Section 10.6).

2.6 Grad-CAM++, LIME and Other Related Methods

Grad-CAM++ [2] keeps the form $\mathrm{ReLU}(\sum_k w_k A^k)$ but derives w_k from higher-order gradient terms, so that several instances of an object or partially visible objects receive better coverage; it re-uses the same A and gradients, so its cost equals Grad-CAM's. We use it as an extra comparison. LIME [13] is model-agnostic: it perturbs superpixels of the image, queries the model many times and fits a sparse linear surrogate. It makes no use of the architecture, but costs hundreds of forward passes per image and depends on the segmentation and sampling; it is discussed here but *not* implemented.

Table 2: Where each syllabus topic stands in this project. "Core" = used by Grad-CAM itself; "Baseline" = implemented and measured here; "Related" = discussed only.

Topic	Role	Comment
Backpropagation / gradient computation	Core	$\partial y^c / \partial A^k_{ij}$ is obtained by reverse-mode autodiff.
CNN feature representations	Core	Feature maps A^k of the last convolutional stage.
Grad-CAM	Core	Re-implemented from scratch.
Saliency maps	Baseline	Vanilla gradient saliency.
CAM	Baseline	Exact for ResNet-50 (GAP + linear head).
Grad-CAM++	Baseline (extra)	Closed-form variant on the same A, G.
LIME	Related	Perturbation-based, not implemented.
Guided backprop / deconvolution	Related	Not implemented; compared conceptually.
t-SNE, UMAP, Mapper, Isomap, LLE, MDS	Outside the paper	Feature-space visualisation, not used by Grad-CAM.

3 Theory and Conceptual Explanation

3.1 CNN Feature Representations

A convolutional stage applies a bank of K learned filters to its input and a non-linearity, yielding K feature maps $A^k \in \mathbb{R}^{h \times w}$. Because convolution and pooling are local and translation-equivariant, position (i, j) of every map still corresponds to a definite (receptive-field) region of the image. Early layers respond to edges and textures; deeper layers combine them into object parts and eventually category-specific patterns.

3.2 Why Convolutional Feature Maps?

A fully connected layer flattens the maps and destroys the spatial layout, so after it nothing says "where". The last convolutional layer is the deepest place where (a) the features are semantically rich and (b) spatial structure is still intact. Going deeper gives more semantics but a coarser grid; going shallower gives a finer grid but less class-specific content. This trade-off is the reason for the layer ablation in Section 10.4.

3.3 Class-Specific Explanation

For a fixed image different classes should give different maps. Grad-CAM achieves this because the weights α^c_k depend on the class c through the gradient of that class's score, while the feature maps A^k are shared. A channel that detects "dog face" receives a large positive weight for the class *dog* and a small or negative one for *cat*.

3.4 Grad-CAM Intuition

Each channel A^k is a detector that says "pattern k occurs here". The classifier then decides how much it cares about pattern k for class c; that importance is α^c_k. The heat-map is the sum of detectors weighted by importance: places where important patterns fire are hot.

3.5 Complete Pipeline

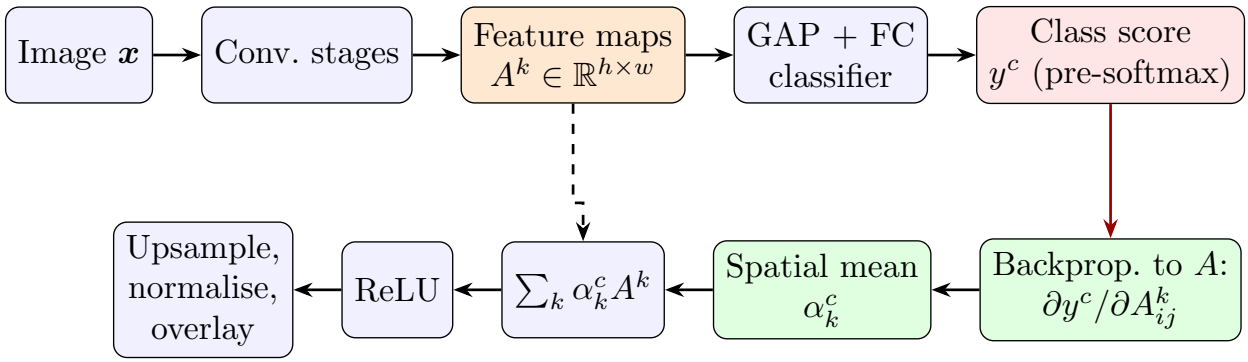


Figure 1: The Grad-CAM pipeline. Top row: ordinary forward pass, the activations A^k of the chosen layer are stored. Bottom row (right to left): the gradient of the class score is propagated back only as far as A, averaged over space to give the channel weights, combined with the stored maps, rectified, and resized to the image.

4 Mathematical Formulation

4.1 Target Class Score

Let $\boldsymbol{x}$ be an image, f the network, and $A^k \in \mathbb{R}^{h\times w}$ $(k = 1, \ldots, K)$ the output of the chosen convolutional layer. Let y^c denote the *pre-softmax* score (logit) of class c. We use the logit rather than the softmax probability for a concrete reason: with $p_c = \mathrm{softmax}(y)_c$,

$$\frac{\partial p_c}{\partial A} = p_c\Big(\frac{\partial y^c}{\partial A} - \sum_j p_j \frac{\partial y^j}{\partial A}\Big), \tag{1}$$

so the gradient of a probability mixes in all other classes and vanishes when $p_c \to 1$ (saturation); the logit gradient isolates evidence for class c alone. In our multi-label VOC model each logit feeds a sigmoid, and the same argument applies.

4.2 Gradient Computation

The quantity

$$g^k_{ij} = \frac{\partial y^c}{\partial A^k_{ij}} \tag{2}$$

is the sensitivity of the class score to a unit change of the activation of channel k at location (i, j). It is obtained by one backward pass that starts at y^c and stops at layer A (reverse-mode differentiation, i.e. backpropagation). To first order, $y^c(A + \Delta) \approx y^c(A) + \sum_{k,i,j} g^k_{ij}\Delta^k_{ij}$, so g^k_{ij} says how much increasing that single activation would raise the class score.

4.3 Importance Weights

Collapsing the spatial dimensions by averaging gives the channel weight

$$\alpha^c_k = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A^k_{ij}}, \qquad Z = h \cdot w. \tag{3}$$

Here Z is the number of spatial positions, α^c_k is the average marginal contribution of channel k to class c, and the averaging is global average pooling of the gradient map. *Why average?* A channel's pattern may occur anywhere; we want one number saying how useful the *pattern* is, independent of position, and the mean is the natural linear aggregate: if the score were linear in the channel's total activation, $y^c \approx \sum_k \alpha^c_k \sum_{ij} A^k_{ij}$, then α^c_k is exactly its coefficient.

Proof that CAM is a special case. Suppose the network ends in GAP followed by one linear layer with weights w_k^c (bias omitted, as it does not depend on A): $y^c = \sum_k w_k^c \frac{1}{Z} \sum_{ij} A_{ij}^k$. Then $\partial y^c / \partial A_{ij}^k = w_k^c / Z$ for every (i, j), hence

$$\alpha_k^c = \frac{1}{Z} \sum_{ij} \frac{w_k^c}{Z} = \frac{w_k^c}{Z}, \qquad L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \frac{w_k^c}{Z} A^k \right) = \frac{1}{Z} \text{ReLU}\left(M^c\right), \tag{4}$$

where $M^c = \sum_k w_k^c A^k$ is the CAM map. So on a CAM-compatible network Grad-CAM equals CAM up to the constant $1/Z$ and a ReLU, which disappears after normalisation. The paper makes the same point; we additionally verify Eq. (4) numerically in `tests/test_gradcam.py` (Section 6). ResNet-50 has exactly this structure after `layer4`, which is why it is a clean testbed for comparing the two.

4.4 Grad-CAM Heat-map

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_{k=1}^{K} \alpha_k^c \, A^k \right) \in \mathbb{R}^{h \times w}. \tag{5}$$

Why a weighted sum of feature maps? Each A^k localises one pattern; the weighted sum is a spatial map of class-relevant pattern evidence, consistent with the linear model above. *Why the final convolutional layer?* It is the deepest layer with intact spatial layout and the richest semantics (Section 3); we test shallower layers in Section 10.4.

4.5 ReLU and Upsampling

ReLU. A negative contribution at (i, j) means the pattern there *lowers* the class score, which typically happens where evidence for *other* classes lies. We want the region that supports class c, so negative values are clipped. The original paper reports that without the ReLU the maps tend to highlight more than the target class [16]. A consequence worth noting: if no location has positive net evidence, the map is identically zero. This does occur for classes the network does not believe are present; we count and report such *empty maps* (Table 4).

Upsampling and normalisation. The map has the size $h \times w$ of the layer (e.g. 14×14 for `layer4` at 448×448 input). It is resized to image resolution by bilinear interpolation, $\tilde{L}^c = \text{Resize}(L_{\text{Grad-CAM}}^c)$, and min–max normalised to $[0, 1]$ for display and thresholding. Bilinear interpolation produces a smooth map; it cannot add detail finer than the grid, which is the origin of Grad-CAM's coarse localisation.

4.6 Interpretation of the Equations

Table 3: Symbols.

Symbol	Meaning
$\boldsymbol{x}$, c	input image, target class
y^c	pre-softmax score of class c
A_{ij}^k	activation of channel k at spatial location (i, j) of the target layer
K, h, w, $Z = hw$	number of channels, map height/width, number of positions
α_k^c	importance of channel k for class c (spatial mean of the gradient)
$L_{\text{Grad-CAM}}^c$	coarse heat-map; $\tilde{L}^c$ its upsampled, normalised version

The chain *CNN representation* → *gradient-based channel importance* → *weighted feature maps* → *ReLU* → *heat-map* corresponds to Eqs. (2), (3), (5) and the resize step.

Complexity. The forward pass costs $O(F)$ (F = network FLOPs). The backward pass runs only from the logits to the target layer, so for the last stage of ResNet-50 it covers just the average-pool and the linear layer: negligible next to F. Shallower targets need back-propagation through the stages above them. The map itself costs $O(Khw)$ and the resize $O(HW)$. Memory: the stored activations $K \cdot h \cdot w$ floats (for `layer4`, $2048 \times 14 \times 14 \approx 0.4$M floats). These are theoretical statements; measured times are in Section 10.5.

5 Algorithm / Methodology

5.1 Input

Image $\boldsymbol{x} \in \mathbb{R}^{3\times H\times W}$ (ImageNet-normalised), a trained network, a target layer, and a class index c (not necessarily the predicted one). **Output**: a heat-map $\tilde{L}^c \in [0,1]^{H\times W}$.

5.2 Forward Pass

Run the network; a forward hook stores A of the target layer without detaching it from the graph.

5.3 Backward Pass

Differentiate the scalar y^c with respect to A (not the parameters) to obtain $G = \partial y^c/\partial A$.

5.4 Heatmap Construction

Compute α by averaging G over space, form $\mathrm{ReLU}(\sum_k \alpha_k A^k)$, resize, and normalise.

5.5 Visualisation and Localisation

For display the map is colour-coded and blended with the image. For *evaluation* the map is converted into a region by the protocol below (Section 7.5).

Algorithm 1 Grad-CAM for image $\boldsymbol{x}$, class c, layer ℓ

1. $y \leftarrow f(\boldsymbol{x})$; store $A \leftarrow$ output of layer ℓ
2. $G \leftarrow \partial y^c/\partial A$ (one partial backward pass)
3. $\alpha_k \leftarrow \frac{1}{Z}\sum_{ij} G^k_{ij}$ for $k = 1..K$
4. $L \leftarrow \mathrm{ReLU}(\sum_k \alpha_k A^k)$
5. $\tilde{L} \leftarrow \mathrm{Resize}(L, (H, W))$; $\tilde{L} \leftarrow (\tilde{L} - \min \tilde{L})/(\max \tilde{L} - \min \tilde{L} + \epsilon)$
6. **return** $\tilde{L}$

Figure 2: Pseudocode of the method as implemented in `src/gradcam.py`.

6 Implementation

6.1 Framework and Environment

PyTorch [11] with torchvision. **Libraries used**: PyTorch/torchvision (autodiff, ResNet-50 definition and ImageNet weights, `VOCDetection` file parsing), SciPy [22] (connected components), scikit-learn [12] (average precision), NumPy/pandas/Matplotlib. **Written for this project**: all explanation methods (`gradcam.py`, `saliency.py`, `cam.py`), the dataset wrapper, the localisation metrics, the evaluation/benchmark/figure scripts and the unit tests. No existing Grad-CAM repository or package (e.g. `pytorch-grad-cam`) was used or consulted for the code. I ran all the experiments on Colab, and the results and conclusions come from those runs.

6.2 Model

ResNet-50 [7] with ImageNet-pretrained weights [15]. We chose it because (i) it is small enough for a free Colab GPU, (ii) after `layer4` it has exactly the GAP→linear structure so that CAM is applicable and the baseline comparison is exact, and (iii) it has three natural stages (`layer2/3/4`, strides 8/16/32) for the layer ablation. VGG-16 was the original paper's main network but needs a modified, retrained head for CAM. The 1000-way head is replaced by a 20-way linear layer (multi-label, sigmoid).

6.3 Dataset Processing

Described in Section 7. Images are resized to 448×448 so that `layer4` yields a 14×14 map (7×7 at the usual 224 input would be very coarse); boxes are rescaled accordingly.

6.4 Grad-CAM Implementation

Capturing activations. A forward hook stores the layer output *with its graph*:

```
def hook(module, inputs, output):
    self._acts[name] = output        # keep the graph: we differentiate w.r.t.
        this tensor
```

Gradients of the class score. We differentiate the logit with respect to the stored activations only; no parameter gradients are computed:

```
score = logits[torch.arange(x.size(0), device=x.device), class_idx].sum()   # y
    ^c, pre-softmax
names = list(self._acts)
grads = torch.autograd.grad(score, [self._acts[n] for n in names])          #
    dy^c / dA^k_ij
```

Weights, weighted sum, ReLU — Eqs. (3) and (5):

```
alpha = G.mean(dim=(2, 3), keepdim=True)              # alpha_k^c = (1/Z)
    sum_ij dy^c/dA^k_ij
cam = F.relu((alpha * A).sum(dim=1, keepdim=True))    # ReLU( sum_k alpha_k^c A
    ^k )
```

Lines 1 and 2 correspond one-to-one to Eqs. (3) and (5); `G.mean(dim=(2,3))` is the spatial average and the broadcasted product followed by `sum(dim=1)` is the weighted sum over channels. The result is resized bilinearly and min–max normalised (`upsample`, `normalize_map`). **Verification.** `tests/test_gradcam.py` checks (a) Eq. (4) numerically on ResNet-50 ($\alpha = w/Z$ and map $= \mathrm{ReLU}(M^c)/Z$), (b) that our `autograd.grad` route gives the same α as an independent `backward()`+`retain_grad()` route at `layer2/3/4`, (c) output range and class-specificity, and (d) the IoU/box code. These tests use random weights and synthetic inputs; they validate the code and produce no experimental results.

6.5 Baseline Implementation

Vanilla saliency (`saliency.py`) differentiates the same logit with respect to the *input*:

```
x = x.clone().requires_grad_(True)
class_idx = torch.as_tensor(class_idx, device=x.device).long().reshape(-1).
    expand(x.size(0))
logits = model(x)
score = logits[torch.arange(x.size(0), device=x.device), class_idx].sum()
(grad,) = torch.autograd.grad(score, x)               # dy^c / dx, same shape
    as the image
sal = grad.abs().amax(dim=1, keepdim=True)            # max over RGB channels
```

CAM (`cam.py`) uses the weights of the final linear layer: $M_{ij}^c = \sum_k w_k^c A_{ij}^k$, min–max normalised without ReLU, as in [25]. **Grad-CAM++** (`gradcam.py`) re-uses A and G with the closed-form weights of [2]. **Centre Gaussian**: a model-free, isotropic Gaussian ($\sigma = 0.25\times$ image size) — a sanity control that exposes how much of the localisation score is explained by objects simply being near the middle of VOC images.

7 Dataset and Experimental Setup

7.1 Dataset

PASCAL VOC 2007 [5, 6]: 20 object categories, images with bounding-box annotations and a *difficult* flag; 5,011 `trainval` and 4,952 `test` images. It is publicly downloadable, annotated with boxes (needed for our metric), and was also used by the original paper's community for weakly-supervised tasks. The images were collected from Flickr; usage terms are given on the dataset website [5]. Image-level labels are derived from the boxes; *difficult* objects are ignored for both labels and boxes.

7.2 Train/Validation/Test Split

`trainval` is split 90%/10% (seeded) into training and validation (used only to pick the checkpoint by mAP); the `test` split is used only for evaluation. Exact counts are listed in Section 7.6.

7.3 Preprocessing

Anisotropic resize to 448×448 (bicubic), ImageNet mean/std normalisation; random horizontal flip during training only.

7.4 Model Configuration and Hyper-parameters

Only `layer4` and the new head are trained; earlier stages and their batch-norm statistics stay frozen. Binary cross-entropy on logits, AdamW [9], learning rates 10^{-3} (head) and 10^{-4} (`layer4`), weight decay 10^{-4}, cosine schedule, batch size 32, 6 epochs, seed 0, mixed precision on GPU (the defaults of `scripts/train.py`, which were used unchanged; they are recorded in `train_log.json`). The explanation experiments do not train anything further.

7.5 Evaluation Metrics

We evaluate the explanation for each ground-truth *(image, class)* pair, using the ground-truth class (not the prediction), so that all methods are scored on identical pairs. For a map $\tilde{L} \in [0,1]^{H\times W}$ and the set $\mathcal{B}_c$ of non-difficult boxes of class c:

1. **Localisation accuracy.** Binarise $\tilde{L} \geq \tau$ (default $\tau = 0.15$ of the maximum, following the weakly-supervised localisation protocol of the original paper (Section 4.1 of the paper: threshold at 15% of the maximum, box around the single largest segment)), keep the largest 4-connected component, take its bounding box B_{pred}, and compute $\text{IoU} = |B_{\text{pred}} \cap B| / |B_{\text{pred}} \cup B|$ against each $B \in \mathcal{B}_c$; the pair counts as correct if $\max_B \text{IoU} \geq 0.5$ (the best match is used because several instances of a class may exist). An empty map has IoU = 0.
2. **Mean IoU** (same quantity, averaged).
3. **Pointing game** [24]: correct if the maximum of $\tilde{L}$ lies inside some box of $\mathcal{B}_c$ (no tolerance; empty maps are failures).
4. **Mass in GT**: $\sum_{p\in\cup\mathcal{B}_c} \tilde{L}(p) / \sum_p \tilde{L}(p)$, a threshold-free measure of how much of the map's energy lies on the object.
5. **Class-discrimination index**: on images with ≥ 2 classes, for classes $c \neq c'$ the mean map value on pixels covered only by c's boxes divided by the sum of that and the mean value on pixels covered only by c''s boxes (0.5 = cannot tell the classes apart).

6. **Cost**: wall-clock per explanation, peak GPU memory, parameter count.

Uncertainty: 95% percentile bootstrap intervals [4] over pairs; differences between methods use a *paired* bootstrap because all methods are scored on the same pairs.

Why these metrics. IoU and localisation accuracy answer "does the highlighted region match the object?" but depend on the threshold and on box shape; the pointing game and mass are insensitive to the threshold; the discrimination index probes the claim that Grad-CAM is class-specific. None of them measures *faithfulness* to the model (deletion/insertion), which we list as a limitation.

7.6 Hardware, Software and Run Facts

- Python 3.13.15
- PyTorch 2.11.0+cu130
- torchvision 0.26.0+cu130
- device: Tesla T4
- train/val images: 4510/501
- best val mAP: 0.9127 (epoch 5)
- test mAP: 0.9178
- test images with labels: 4952
- image–class pairs: 7013

8 Quantitative Results

All numbers in this section were measured in this project (VOC2007 test, all 4,952 images, 7,013 image–class pairs, one run, seed 0) and are read from the files written by `evaluate.py`, `benchmark.py`, `make_tables.py` and `extra_analysis.py`; none is taken from the original paper. The fine-tuned classifier reaches a test mAP of 0.918 (Section 7.6), so most ground-truth classes are recognised and the explanations are of a competent model.

Reading Table 4. At the default threshold $\tau = 0.15$ Grad-CAM localises best: 38.8% of the pairs reach IoU ≥ 0.5 (95% CI 37.7–40.0), against 33.0% for Grad-CAM++, 28.3% for CAM, 23.1% for the model-free centre Gaussian and 0.6% for vanilla saliency; mean IoU follows the same order (0.423, 0.379, 0.348, 0.304, 0.038). The paired bootstrap (Table 5) excludes zero for every localisation-accuracy comparison: Grad-CAM is 5.8 points above Grad-CAM++ (CI +4.9 to +6.9), 10.6 above CAM (CI +9.5 to +11.6) and 15.7 above the centre prior. The pointing game tells a different story: Grad-CAM (85.6%), Grad-CAM++ (85.9%) and CAM (85.6%) are statistically indistinguishable (paired differences -0.3 $[-0.8, +0.2]$ and -0.0), and all are about 22 points above the centre control (63.1%). Grad-CAM and CAM disagree on exactly one of 7,013 pairs, and in that pair the Grad-CAM map has zero energy inside the ground-truth boxes (`extra.json`). This is what Eq. (4) predicts: the two maps have the same arg-max wherever Grad-CAM is non-empty.

Why do the box metrics separate methods that point to the same place? The answer is in the size of the predicted box (Table 6). At $\tau = 0.15$ the CAM box has a median area of 94.4% of the image and exceeds 90% of the image in 52.7% of the pairs, almost like the centre prior (median 94.7%), whereas the Grad-CAM box has a median of 39.9%. Our CAM is min–max normalised without a ReLU (as in the CAM paper), so its zero level sits above 0 after normalisation and a 15% threshold admits most of the image; Grad-CAM clips negative evidence to 0 first, so 15% of its maximum is a much tighter region. Two observations are consistent with this reading: the CAM and Grad-CAM++ accuracy curves peak at higher thresholds ($\tau = 0.3$) than Grad-CAM's ($\tau = 0.2$; Table 12), and CAM's share of heat-map energy inside the ground-truth boxes (48.2%) is lower than Grad-CAM's (64.1%). We did not run a separate control that changes only the

Table 4: Weakly-supervised localisation on VOC2007 test (4952 images, 7013 image–class pairs; threshold 0.15 of the map maximum; brackets: 95% bootstrap CI). Loc. acc. = fraction of pairs with box IoU ≥ 0.5; Mass = share of heat-map energy inside ground-truth boxes; Empty = map identically zero (counted as failure); time = ms per explanation (batch size 1, measured separately, see Table 13).

Method	Loc. acc.@0.5 (%)	Mean IoU	Pointing (%)	Mass in GT (%)	Empty maps (%)	Time (ms)
Grad-CAM (layer4)	38.8 [37.7, 40.0]	0.423	85.6	64.1	0.2	19.2
Grad-CAM++ (layer4)	33.0 [31.9, 34.0]	0.379	85.9	53.2	0.0	20.0
CAM (layer4)	28.3 [27.2, 29.2]	0.348	85.6	48.2	0.0	19.7
Vanilla saliency	0.6 [0.4, 0.8]	0.038	68.5	46.2	0.0	69.2
Centre Gaussian (no model)	23.1 [22.2, 24.1]	0.304	63.1	43.7	0.0	–

Table 5: Paired bootstrap differences (Grad-CAM layer4 minus method) at threshold 0.15, in percentage points.

Comparison	Metric	Diff.	95% CI
Grad-CAM++ (layer4)	loc_acc@0.5	+5.8	[+4.9, +6.9]
Grad-CAM++ (layer4)	pointing	-0.3	[-0.8, +0.2]
CAM (layer4)	loc_acc@0.5	+10.6	[+9.5, +11.6]
CAM (layer4)	pointing	-0.0	[-0.0, +0.0]
Vanilla saliency	loc_acc@0.5	+38.2	[+37.1, +39.4]
Vanilla saliency	pointing	+17.1	[+15.9, +18.2]
Centre Gaussian (no model)	loc_acc@0.5	+15.7	[+14.6, +16.9]
Centre Gaussian (no model)	pointing	+22.5	[+21.3, +23.8]
Grad-CAM (layer2)	loc_acc@0.5	+26.7	[+25.5, +27.8]
Grad-CAM (layer2)	pointing	+35.0	[+33.8, +36.4]
Grad-CAM (layer3)	loc_acc@0.5	+9.3	[+8.2, +10.5]
Grad-CAM (layer3)	pointing	+12.7	[+11.6, +13.9]

normalisation, so this is a well-supported explanation rather than an isolated test. Conversely, vanilla saliency produces extremely peaked maps: its box at $\tau = 0.15$ covers on average 0.4% of the image, hence IoU ≈ 0 and 0.6% accuracy. This number reflects our simple protocol (threshold plus largest component, no smoothing or grab-cut as in the baseline of the original paper), not an intrinsic uselessness of gradient saliency; its pointing accuracy of 68.5% [67.5, 69.7] is above the centre control (63.1%) but only by 5 points.

Centre control. A Gaussian that ignores the image already reaches 23.1% localisation and 63.1% pointing accuracy, because VOC objects are large (median ground-truth region 27% of the image, mean 33%) and often centred. Any claim of a method's quality must therefore be read relative to this control, not relative to zero; in particular, Section 10.3 shows that for objects covering more than 60% of the image the centre control beats Grad-CAM.

Empty maps. Grad-CAM returned an identically zero map for 0.2% of the pairs (16 pairs); Grad-CAM++ and CAM never did. A further set of pairs have a non-empty map with exactly zero energy inside the object boxes (69 Grad-CAM pairs in total with zero mass, all with classifier probability for the class below 0.13; Table 9).

Table 6: Size of the predicted box at $\tau = 0.15$ as a percentage of the image area (ground-truth boxes cover on average 33.3% of the image, median 27.0%). "> 90%" = share of pairs whose box covers more than 90% of the image.

Method	Median area (%)	Mean area (%)	> 90% of image (%)	Loc. acc. (%)
Grad-CAM	39.9	43.7	4.8	38.8
Grad-CAM++	73.7	68.7	22.7	33.0
CAM	94.4	79.6	52.7	28.3
Saliency	0.3	0.4	0.0	0.6
Centre	94.7	94.7	100.0	23.1

Table 7: Class-discrimination index on images that contain ≥ 2 classes (0.5 = the map cannot tell the classes apart; 1 = all heat on the explained class' own box).

Method	# pairs	Index [95% CI]
Grad-CAM (layer4)	3644	0.850 [0.844, 0.856]
Grad-CAM++ (layer4)	3644	0.658 [0.654, 0.662]
CAM (layer4)	3644	0.731 [0.727, 0.735]
Vanilla saliency	3644	0.664 [0.660, 0.668]
Centre Gaussian (no model)	3644	0.500 [0.494, 0.506]

Class discrimination. On the 3,644 ordered class pairs from images with at least two annotated classes, the Grad-CAM map of class c puts clearly more heat on pixels that belong only to c than on pixels that belong only to the other class (index 0.850, CI 0.844–0.856). The centre Gaussian scores 0.500 [0.494, 0.506], which validates the index (a map with no class information is at chance). CAM reaches 0.731, vanilla saliency 0.664 and Grad-CAM++ 0.658. Since our CAM differs from Grad-CAM only by the ReLU (and the resulting zero-anchored normalisation), the 12-point gap quantifies what the rectification removes, in the direction the original paper describes (negative evidence belongs to other classes; Section 3 of [16]). This is an automatic proxy based on box overlap, not the human class-discrimination study of the paper.

Expectation stated before measuring, and the outcome. We expected (i) Grad-CAM and CAM to be close on ResNet-50 because of Eq. (4), (ii) saliency to score lowest on box-IoU, and (iii) high-resolution maps to do relatively better on pointing than on box-IoU. (ii) was confirmed. (i) was only partly right: the arg-max (pointing) is identical, but the thresholded metrics differ by 10.6 points because normalisation changes which region survives the threshold, so "close" holds for *where* the evidence peaks and not for the *extent* of the box. (iii) holds for saliency (68.5% pointing against 0.6% box accuracy).

9 Qualitative Results

Examples are chosen by fixed rules with a fixed random seed (`scripts/generate_heatmaps.py`), not by hand; the rules and the number of pairs that satisfy them are in the captions. Each row shows the image with the ground-truth boxes of the explained class (green; other classes yellow), the Grad-CAM heat-map, the overlay with the predicted box (red), and Grad-CAM++, CAM and vanilla saliency. Descriptions below are of what is visible in these specific rows; they illustrate

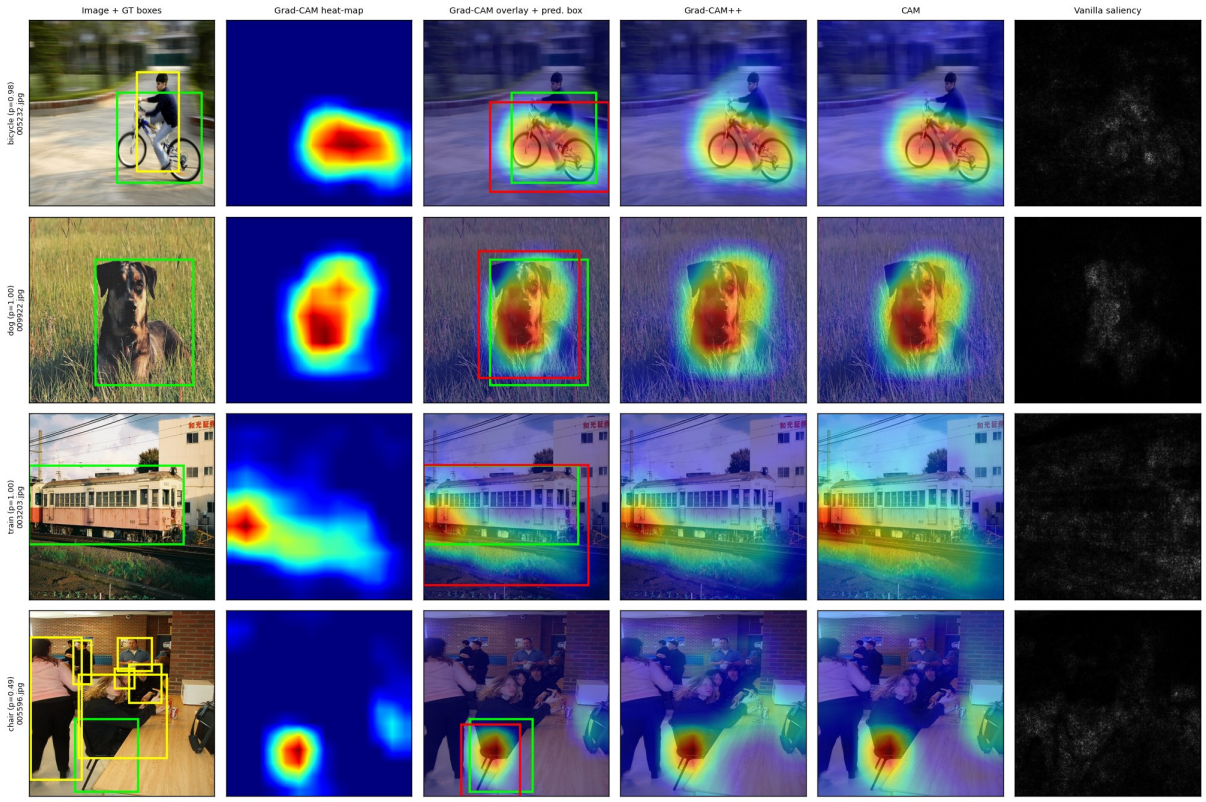


Figure 3: *Successful explanations*: random sample from the 2,624 pairs (37.4%) with IoU ≥ 0.5 and a pointing hit at $\tau = 0.15$. Rows: bicycle, dog, train, chair.

patterns and do not by themselves prove them (the quantitative tables do).

Successes (Figure 3). In the bicycle and dog rows the hot region covers the object body (frame and wheels; head and chest) and little of the surroundings; in the train row the strongest response is at the front end of the train with a weaker band along the carriage, and the red box also spills below onto the track and to the right (it still passes the IoU test); in the chair row the peak is on the seat and legs of the chair (classifier probability 0.49) but a second weaker region appears at the right edge. Grad-CAM++ and CAM overlays are visually very similar to each other and broader than Grad-CAM's, in line with their larger boxes (Table 6). Saliency is a diffuse speckle within and around the object that cannot be turned into a region by a threshold.

Partial explanations (Figure 4). The hot region sits on the most characteristic part: the cyclist's helmet and upper body (person), the face of the dog, and the pot (soil and rim) rather than the foliage of the potted plant; for the chair (probability 0.02) only a small spot near the seat survives while the CAM overlay is diffusely warm over the whole picture. The ground-truth boxes cover the entire object (or nearly the entire image), so these boxes score poorly even though the model is plausibly attending to the right object part.

Background and context (Figure 5). The first bottle (probability 0.02) has its hot spot next to, not inside, the annotated object. In the second row the strongest regions are the lower-left corner (a small white step stool) and the right edge of the armchair rather than the annotated armchair body; in the third row the heat covers the dinner-table scene with plates and food around the thin bottle boxes; in the last row the legs of a large black table and a sofa light up while the annotated chair is a small object far behind. A plausible reading is that the model uses co-occurring context or look-alike furniture (a stool for a chair, a dinner table for bottles); we did not test this

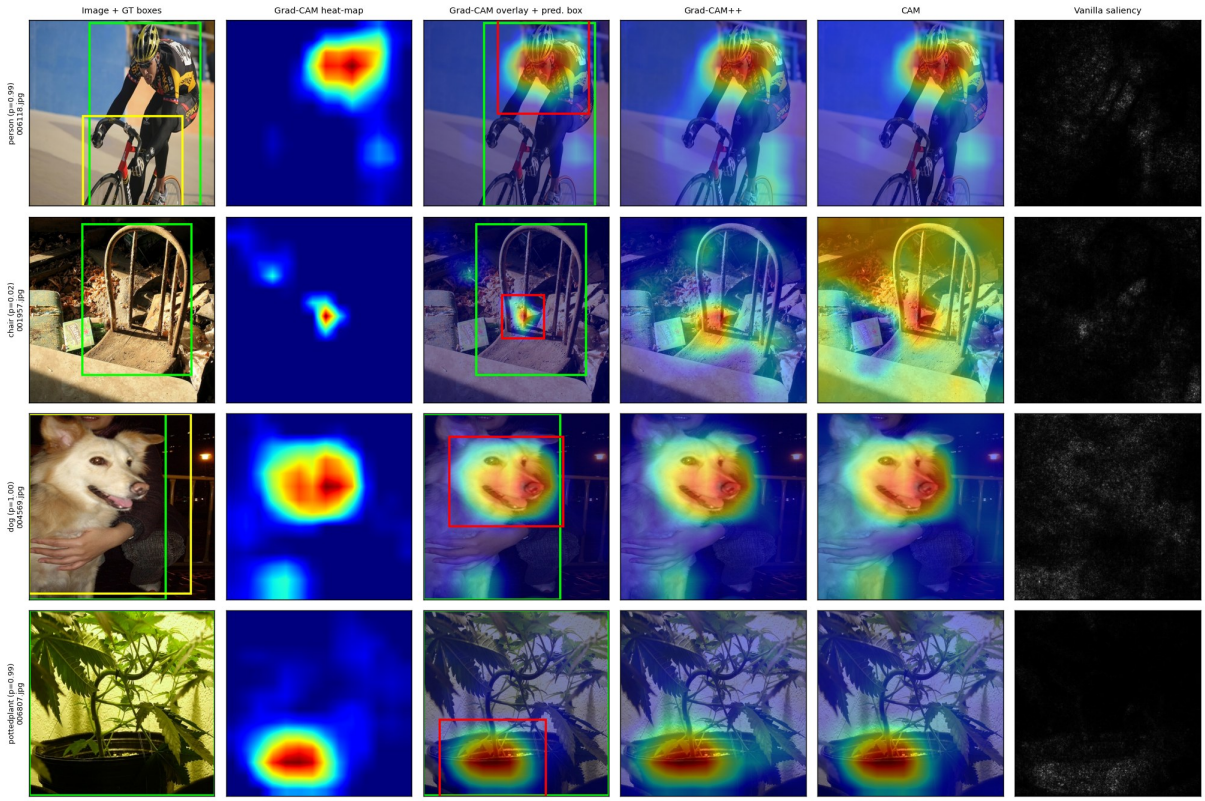


Figure 4: *Partial explanations*: random sample from the 375 pairs (5.3%) with a pointing hit but a predicted box smaller than half of the ground-truth area. Rows: person, chair, dog, potted plant.

with interventions, so it remains a hypothesis.

Misclassification (Figure 6). The explanation often makes the error understandable: for "potted plant" (0.10) the peak is on the greenery behind the pool; for "person" (0.45) it is on bare feet and legs; for "tv monitor" (1.00) it is on the television screen, which visibly shows a person. The last row is probably not a model error at all: the television is present but not counted as a ground-truth class, likely because VOC marks it *difficult* and our labels ignore difficult objects (not checked in the annotation file). Our "top-1 not in ground truth" rule therefore also catches annotation artefacts and low-confidence images (the top class has probability only 0.10 in the first row).

Multiple classes (Figure 7). In the first image the motorbike map is concentrated on the lower and central part of the bike while the person map is on the upper right where the people's heads and torsos are; in the second image the motorbike map is scattered over several regions (left edge, handlebar area, rear wheel) while the person map is a single horizontal band around the small distant figure. The maps for the two classes are clearly different, as the discrimination index (Table 7) suggests quantitatively; they are not perfect (the motorbike map in the second image is fragmented).

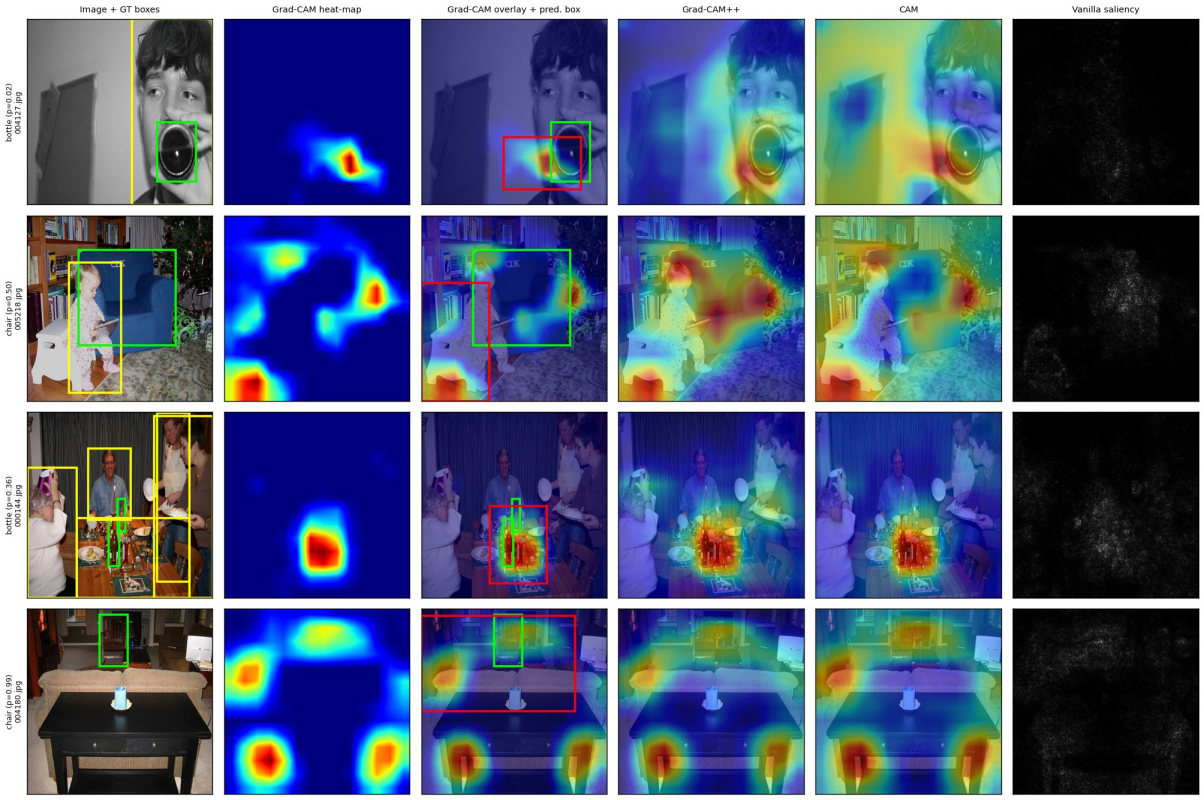


Figure 5: *Background / context dependence*: random sample from the 559 pairs (8.0%) with a pointing miss and less than 30% of the map's energy inside the ground-truth boxes. Rows: bottle, chair, bottle, chair.

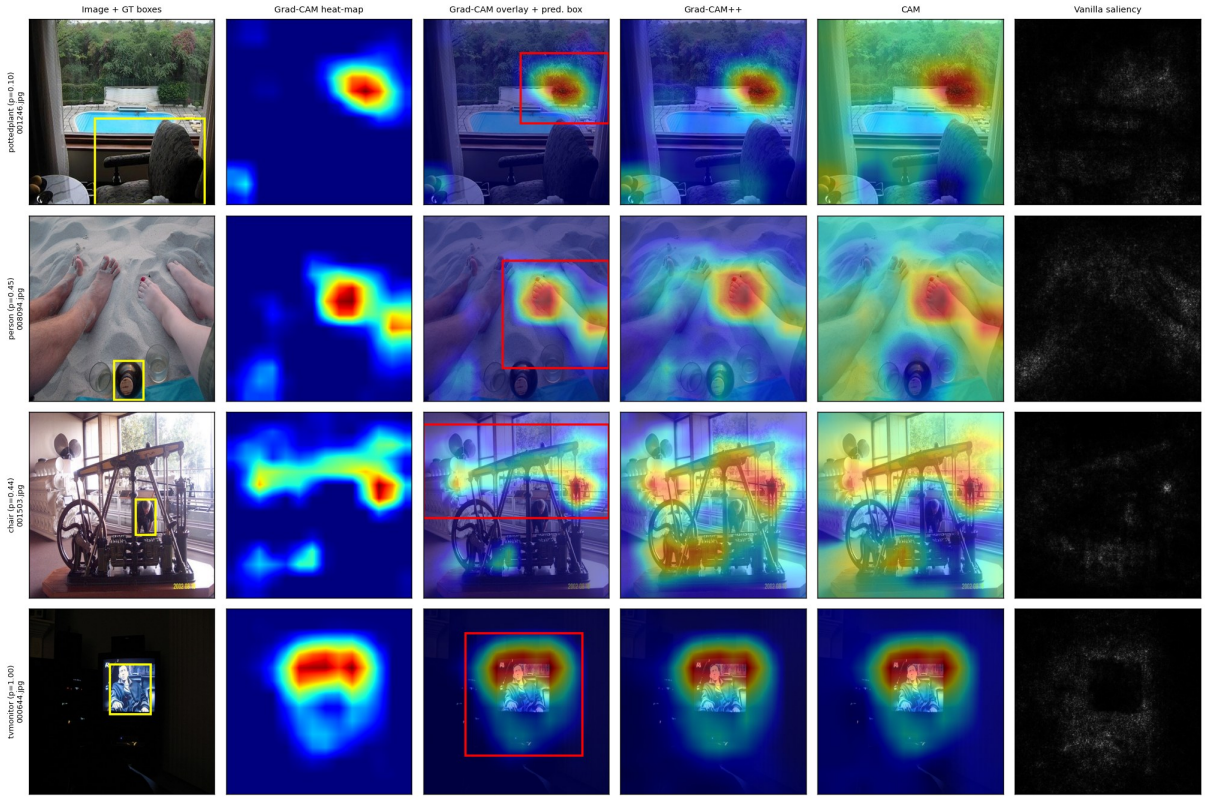


Figure 6: *Misclassification*: random sample of images whose top-1 predicted class is absent from the ground truth; the maps explain the *predicted* class (probability in the row label). Rows: potted plant, person, chair, tv monitor.

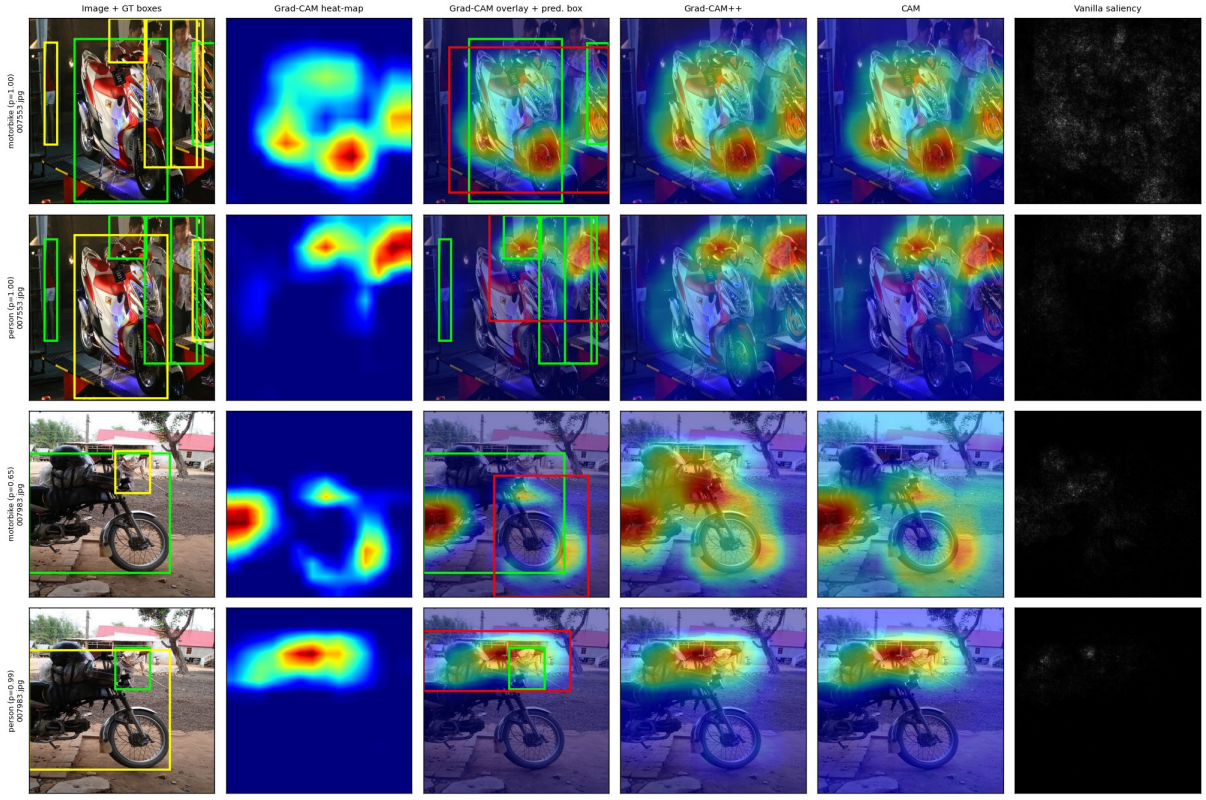


Figure 7: *Several classes in one image*: two images, each explained for two of its classes in consecutive rows (motorbike and person in both).

10 Analysis and Discussion

10.1 Comparison with Baseline

Grad-CAM is the best of the five methods on every box-based metric at the default threshold, but the margin has three different sources that should not be confused. (1) *Against saliency* the difference is large and real for pointing too (+17.1 points, CI +15.9 to +18.2): a one-step gradient with respect to pixels is local and noisy, whereas Grad-CAM aggregates a semantically rich layer; the box-metric gap (+38.2) is exaggerated by our naive region extraction. (2) *Against CAM* the pointing result is identical, as the derivation predicts, and the box gap (+10.6) is explained by the ReLU and zero-anchored normalisation, which shrink Grad-CAM's region, not by better evidence. (3) *Against Grad-CAM++* the pointing difference is not significant either, and the box gap (+5.8) again comes mainly from region size at a fixed τ. When each method is allowed its own best threshold (Table 12) the three CAM-family methods reach 39.4% (Grad-CAM, $\tau = 0.2$, CI 38.4–40.5), 39.7% (Grad-CAM++, $\tau = 0.3$, CI 38.5–40.7) and 38.3% (CAM, $\tau = 0.3$, CI 37.2–39.4), whose confidence intervals overlap almost entirely. Because these thresholds were picked on the test set the values are optimistic and no paired test was run at the best thresholds, so the safe conclusion is: *we see no evidence that one of the three gives better object extents than the others once the threshold is calibrated; Grad-CAM's advantage at $\tau = 0.15$ is mostly a calibration effect.* The advantage over the centre control (+15.7 points localisation, +22.5 pointing) is genuine, but it is concentrated in small and medium objects (Table 8).

10.2 Successful Examples

In the 2,624 successes (37.4% of pairs; Figure 3) the classifier is confident and the object is mid-sized to large. Success rises steeply with classifier confidence (44.5% localisation accuracy when the class probability is ≥ 0.9, Table 9) and with object size. Per class (Table 10) Grad-CAM does best on cat (63.0%), sofa (57.8%), dog and dining table (57.4%) and horse (56.6%): compact, large, distinctive objects.

10.3 Failure Cases

We distinguish observed facts (from tables and figures) from hypotheses.

1. **Small objects and the coarse grid.** For objects covering less than 10% of the image, Grad-CAM reaches 3.9% localisation accuracy but 67.8% pointing accuracy, far above the centre control's 18.2% (Table 8). The predicted box is on average 21.0% of the image for objects whose ground truth is 4.5% (measured), i.e. it finds the object but the region is several times too large. *Hypothesis*: with a 14×14 grid at layer4 one cell is about 0.5% of the image and bilinear upsampling spreads each activation over its neighbours, so small objects cannot be delineated; layer3 (28×28) was nevertheless worse overall (Section 10.4), so resolution alone does not fix it.
2. **Large objects: the box under-fills, and a no-model prior wins.** For objects covering more than 60% of the image the Grad-CAM box averages 67.6% of the image versus a ground truth of 76.5%, and localisation accuracy is 67.7% against 84.1% for the centre prior, 83.9% for CAM and 84.2% for Grad-CAM++ (Table 8). Methods that output a near-full-image box win here by construction. The overall ranking is thus partly a statement about the size mix of VOC, which is a limitation of box-IoU as a metric.
3. **Pointing is right, region is wrong.** In 3,381 pairs (48.2%) the Grad-CAM peak lies inside a ground-truth box but IoU is below 0.5. This is the dominant failure of the box metric and combines patterns 1 and 2 with partial coverage of the object (Figure 4: the helmet, the dog's face and the pot are discriminative parts; 375 pairs, 5.3%, satisfy the strict "box under half the ground truth" rule). *Hypothesis*: the classifier relies on the most discriminative part, and the gradient weights follow that part.
4. **The classifier does not recognise the class.** Localisation accuracy falls from 44.5% (probability ≥ 0.9) to 30.9% (0.5–0.9), 21.9% (0.1–0.5) and 11.5% (< 0.1); pointing falls from

Table 8: Localisation accuracy (IoU ≥ 0.5, %) by relative size of the ground-truth region (union of the class' boxes as a share of the image), and pointing accuracy (%) for Grad-CAM and the centre control, $\tau = 0.15$.

GT size	# pairs	Grad-CAM	Grad-CAM++	CAM	Centre	Point: Grad-CAM	Point: Centre
$< 10\%$	1787	3.9	0.4	0.1	0.0	67.8	18.2
10–30%	1968	29.7	7.7	4.1	0.0	85.6	55.0
30–60%	1876	60.4	52.8	39.4	24.5	93.9	88.1
$> 60\%$	1382	67.7	84.2	83.9	84.1	97.4	98.6

Table 9: Grad-CAM (layer4, $\tau = 0.15$) by the classifier's probability for the ground-truth class being explained. "Zero mass" = pairs where the map has exactly zero energy inside the ground-truth boxes.

p(class)	# pairs	Loc. acc. (%)	Pointing (%)	Mass (%)	Zero mass
< 0.1	496	11.5	56.5	42.0	68
0.1–0.5	588	21.9	76.0	52.6	1
0.5–0.9	754	30.9	84.2	58.2	0
≥ 0.9	5175	44.5	89.7	68.4	0

89.7% to 56.5% (Table 9). 1,084 of the 7,013 pairs have probability below 0.5. Of the 69 pairs with zero mass inside the object, 68 are in the lowest bin. Mechanistically, Grad-CAM reports the evidence the network *has* for the class; when the network sees no evidence the map is empty (16 pairs) or points elsewhere, which is the correct behaviour for an explanation but counts as a failure for an object-localisation metric.

5. **Context and look-alikes.** 559 pairs (8.0%) are pointing misses with little energy on the object (Figure 5); chair, bottle and tv monitor have the lowest accuracy (10.6%, 11.3%, 14.0%; Table 10), and also the smallest ground-truth regions (22%, 12%, 17% of the image on average), so size and context are confounded. *Hypothesis* from the figures: furniture and dinner-table context drive these classes; we did not run an occlusion or counterfactual test to separate context from size.
6. **Multiple instances (hypothesis only).** Person has 2,007 pairs (29% of all) and 30.3% accuracy. We extract a single connected component as the box, while the best-IoU is taken over instances; a map covering several people therefore yields one merged or one partial box. We did not measure this directly.
7. **Does the target class change the map? Yes.** In Figure 7 the same image gives visibly different maps for motorbike and person, and the discrimination index is 0.850 against 0.500 for a class-blind map.

10.4 Sensitivity / Ablation Analysis

Two analyses were performed: the target layer (`layer2/3/4`) and the binarisation threshold. An input-perturbation study (occlusion, noise) was not performed.

Target layer. Localisation accuracy increases monotonically with depth, from 12.1% (layer2, 56×56 map) through 29.5% (layer3, 28×28) to 38.8% (layer4, 14×14); pointing rises from 50.6% to 72.9% to 85.6%, and the share of energy on the object from 36.9% to 44.7% to 64.1%. Layer4 is significantly better than both shallower layers in the paired test (Table 5; layer4 minus layer2: $+26.7$ $[+25.5, +27.8]$, minus layer3: $+9.3$ $[+8.2, +10.5]$). Layer2 is *below* the model-free centre control on pointing (50.6% vs. 63.1%) and on energy on the object (36.9% vs. 43.7%): its maps carry little class-specific information. Figure 8 shows why: at layer2 the map is a speckle over

Table 10: Per-class results ($\tau = 0.15$): localisation accuracy (%) of Grad-CAM, CAM and the centre control, Grad-CAM pointing accuracy (%), and mean ground-truth region size (% of image).

Class	# pairs	Loc: Grad-CAM	Loc: CAM	Loc: Centre	Point: Grad-CAM	GT size
aeroplane	204	49.5	37.3	26.0	91.2	34
bicycle	239	54.0	28.9	23.8	95.0	33
bird	282	31.9	25.2	20.9	82.6	31
boat	172	22.7	20.3	19.2	70.9	27
bottle	212	11.3	4.7	2.8	66.5	12
bus	174	48.9	35.1	29.3	88.5	38
car	721	49.2	38.7	30.0	87.9	34
cat	322	63.0	53.4	46.3	95.7	47
chair	417	10.6	10.3	11.0	71.5	22
cow	127	37.8	24.4	21.3	88.2	31
diningtable	190	57.4	27.9	27.4	87.4	38
dog	418	57.4	45.7	37.8	95.2	42
horse	274	56.6	34.7	25.2	92.3	38
motorbike	222	55.9	40.5	33.3	95.0	40
person	2007	30.3	23.1	17.1	83.0	34
pottedplant	224	24.1	16.1	14.3	81.2	24
sheep	97	24.7	15.5	14.4	88.7	29
sofa	223	57.8	32.3	32.3	92.4	39
train	259	50.2	39.4	35.9	87.6	39
tvmonitor	229	14.0	7.9	7.9	85.2	17

Table 11: Layer-selection ablation (ResNet-50, input 448×448, threshold 0.15).

Target layer	Map size	Loc. acc. (%)	Mean IoU	Pointing (%)	Mass (%)
Grad-CAM (layer2)	56×56	12.1	0.197	50.6	36.9
Grad-CAM (layer3)	28×28	29.5	0.355	72.9	44.7
Grad-CAM (layer4)	14×14	38.8	0.423	85.6	64.1

background texture such as the grass around the dog, at layer3 object parts (wheels, dog face) appear alongside many background spots, and layer4 gives a compact blob on the object. So finer resolution does not compensate for weaker semantics, in agreement with the trade-off argued in Section 3 and with the statement of the original paper that the discriminative ability of Grad-CAM reduces going to shallower layers of ResNets [16]. (We measured this on classification localisation; the paper's statement concerns a different visualisation.)

Threshold. The optimum depends strongly on the method: Grad-CAM peaks at $\tau = 0.2$ (39.4%) and is fairly flat between 0.1 and 0.3 (37.1–39.4%); CAM and Grad-CAM++ peak at $\tau = 0.3$ (38.3% and 39.7%) and are much worse at 0.05–0.15; the centre control also peaks at 0.3 (29.1%); saliency is best at the smallest threshold (15.2% at 0.05) and collapses afterwards. The ranking of the methods therefore changes with τ: Grad-CAM leads at $\tau \leq 0.2$; from $\tau = 0.3$ on Grad-CAM++ and CAM are ahead of it (at $\tau = 0.3$: 39.7% and 38.3% against 37.1%; at $\tau = 0.5$: 31.8% and 33.6% against 24.6%). Any statement "method A localises better than B" based on a single threshold should be treated with caution.

10.5 Computational Analysis

Measured on a Tesla T4 at 448×448, batch size 1, full precision, 100 timed runs after warm-up: Grad-CAM at layer4 costs 19.2 ms against 18.8 ms for the plain forward pass (+0.4 ms, about

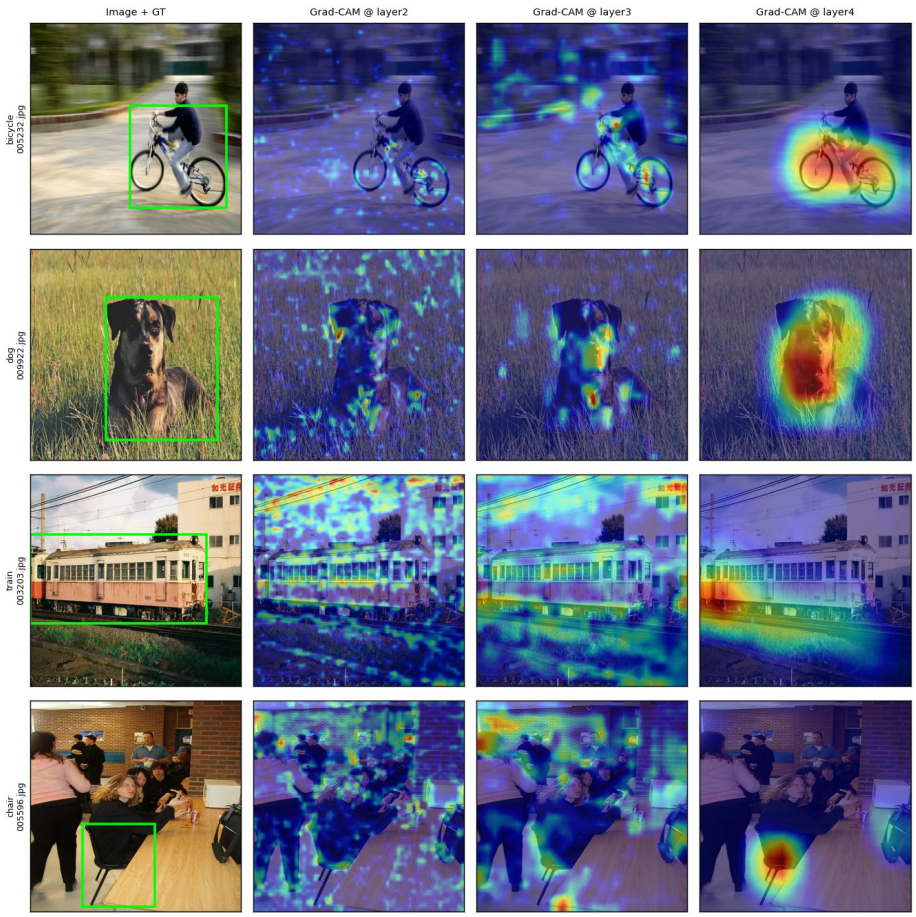


Figure 8: Grad-CAM at `layer2`, `layer3` and `layer4` for the four successful pairs of Figure 3 (bicycle, dog, train, chair). Resolution increases toward the left, semantic specificity toward the right.

2%), because the backward pass starts at the logit and stops at the layer4 output, crossing only the pooling and the linear layer. Targeting layer2 requires back-propagating through layer3 and layer4 and costs 32.0 ms (+70%). Grad-CAM++ adds about 0.8 ms to Grad-CAM, CAM needs 19.7 ms, and vanilla saliency, which back-propagates to the input, needs 69.2 ms (3.7 times the forward pass). The model has 23,549,012 parameters. Peak GPU memory is 243 MB for a forward pass but 705 MB for Grad-CAM at layer4 (788 MB at layer2) and only 257 MB for CAM. This is higher than the theoretical activation storage ($K \cdot h \cdot w \approx 0.4$M floats) discussed in Section 4; *hypothesis*: because our implementation lets autograd record the whole forward pass (it makes the input require gradients), intermediate activations are kept. A variant that differentiates only the last stage from a detached activation would likely need less memory; we did not test it. FLOPs were not measured. In practice Grad-CAM at the last stage is almost free relative to inference, which is a strong deployment argument over perturbation methods such as LIME that need hundreds of forward passes (not measured here).

10.6 Limitations

From the literature. (i) The map has the grid size of the target layer, so fine boundaries and small objects are poorly delineated. (ii) A highlighted region is not proof that the model *used* it: Adebayo et al. [1] urge sanity checks for saliency methods, and Draelos and Carin [3] propose HiResCAM because, they argue, the spatial averaging of gradients can make the map unfaithful to the model's actual computation in some architectures. (iii) Results depend on the chosen layer. (iv) The explanation reflects the classifier's evidence, which may be wrong. **Observed here.** (a)

Table 12: Sensitivity to the binarisation threshold: localisation accuracy (IoU ≥ 0.5, %) of each method.

Method	0.05	0.1	0.15	0.2	0.3	0.4	0.5	0.6
Grad-CAM (layer4)	35.0	37.3	38.8	39.4	37.1	32.0	24.6	17.4
Grad-CAM++ (layer4)	24.1	28.6	33.0	36.0	39.7	38.4	31.8	23.3
CAM (layer4)	22.2	24.6	28.3	32.4	38.3	38.2	33.6	25.1
Vanilla saliency	15.2	3.5	0.6	0.1	0.0	0.0	0.0	0.0
Centre Gaussian (no model)	22.0	22.0	23.1	25.9	29.1	27.5	18.5	11.4

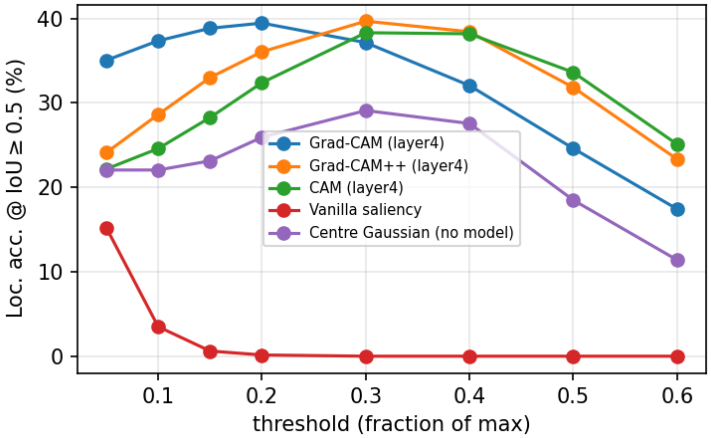


Figure 9: Localisation accuracy as a function of the threshold τ for each method (measured, VOC2007 test).

Localisation scores depend heavily on the threshold and on normalisation (Tables 12,6); (b) box-IoU is confounded by object size and VOC's centre bias, since a model-free Gaussian scores 23.1% and beats Grad-CAM on objects covering more than 60% of the image; (c) low-confidence classes yield empty or off-object maps; (d) the coarse 14×14 map is too large for small objects. **Limits of our protocol.** One architecture and dataset, one training run and seed, boxes instead of masks, no deletion/insertion or occlusion faithfulness test, no human study, thresholds for the best-case comparison chosen on the test set, saliency scored with a deliberately simple region extraction, and a misclassification rule that also catches annotation artefacts.

10.7 Comparison with the Original Paper

Original-paper results (Table 1 and Section 4.1 of [16]). The authors evaluate weakly-supervised localisation on the ILSVRC-15 validation set with an off-the-shelf VGG-16: they generate maps for the predicted classes, binarise at 15% of the maximum, and draw a box around the single largest connected segment; localisation error is reported for the top-1 and top-5 predicted categories. We use the same 15% threshold and largest-component rule, but everything else differs. **Key differences.** (1) Dataset: single-label ImageNet versus multi-label VOC07. (2) Network: VGG-16 for Grad-CAM and a *separately retrained* VGG-16-GAP for CAM in the paper, versus one ResNet-50 whose native head is already GAP→linear, so our CAM and Grad-CAM share the model and differ only by the ReLU and normalisation. (3) Metric: the ILSVRC error counts a prediction as wrong if the class is wrong *or* the box has IoU below 0.5; we explain the ground-truth classes, so classification errors do not enter. (4) The paper's saliency baseline uses grab-cut post-processing; ours does not. **What we can and cannot say.** The paper reports Grad-CAM slightly ahead of CAM on top-1 localisation error (56.51 vs. 57.20) and CAM ahead on top-5 (45.14 vs. 46.41). In

Table 13: Measured cost per image (batch 1, 448×448 input, 100 runs after warm-up, device: Tesla T4). Parameters: 23,549,012.

Method	Time (ms)	Peak GPU mem. (MB)
forward only	18.8 ± 0.3	243
Grad-CAM (layer4)	19.2 ± 0.2	705
Grad-CAM (layer2)	32.0 ± 0.4	788
Grad-CAM++ (layer4)	20.0 ± 0.3	711
CAM (layer4)	19.7 ± 0.3	257
Vanilla saliency	69.2 ± 0.6	548

Table 14: Original-paper numbers versus ours. Left block: values *copied from Table 1 of the paper* (localisation *error* in %, lower is better). Right block: *our measurements* (localisation *accuracy* in %, higher is better). The two blocks use different datasets, networks and metrics and must not be compared numerically.

	Paper: ILSVRC-15 val, VGG-16		Ours: VOC07 test, ResNet-50	
Method	Top-1 loc. err.	Top-5 loc. err.	Acc. $\tau = 0.15$	Acc. best τ
Grad-CAM	56.51	46.41	38.8	39.4 ($\tau = 0.2$)
CAM (VGG-16-GAP, retrained)	57.20	45.14	28.3 (same network)	38.3 ($\tau = 0.3$)
Gradient saliency (Simonyan et al., with grab-cut)	61.12	51.46	0.6 (no grab-cut)	15.2 ($\tau = 0.05$)

our experiment Grad-CAM is ahead of CAM at $\tau = 0.15$ in the same direction as the top-1 result but by a much larger margin (10.6 points), which we traced to normalisation (Table 6); at each method's best threshold the gap shrinks to about 1 point with overlapping intervals, closer in spirit to the paper's small gap. We cannot claim that this reproduces the paper's numbers. What we do reproduce, as ideas: (a) Grad-CAM yields usable object localisation from a trained classifier without any change to the architecture; (b) maps for different target classes differ (index 0.850 vs. 0.500 for a class-blind map), supporting the class-discrimination claim of the paper's Figure 1 only qualitatively, since we did not run its human study (which reports 61.23% for Guided Grad-CAM against 44.44% for Guided Backpropagation); (c) Grad-CAM reduces to CAM on a GAP network (Eq. (4); numerical test, identical pointing). A small note on notation: the paper states $\alpha_k^c = w_k^c$ for such networks while Eq. (3) gives w_k^c/Z; the constant Z is removed by normalisation. We did *not* reproduce the paper's faithfulness experiment (rank correlation with occlusion maps, 0.254 for Grad-CAM on VOC 2007 val) or its captioning, VQA and bias-diagnosis studies.

11 Conclusion

We derived Grad-CAM, showed analytically and numerically that it equals CAM up to a constant on a GAP network, re-implemented it from scratch, and evaluated it on all of VOC2007 test with a fine-tuned ResNet-50 (test mAP 0.918). Measured findings: (1) Grad-CAM localises the explained class with 85.6% pointing accuracy and 38.8% box accuracy at $\tau = 0.15$, well above a model-free centre prior (63.1% and 23.1%), with the pointing performance matching CAM and Grad-CAM++; (2) the box-metric advantage over CAM and Grad-CAM++ at the default threshold is mainly a threshold/normalisation effect and vanishes when each method gets its own threshold; (3) deeper layers are much better than shallow ones (layer2 is worse than the centre prior on pointing), so resolution does not compensate for semantics; (4) Grad-CAM at the last stage costs about 2% extra time over inference, versus 3.7× for saliency; (5) failures are dominated by small objects (box

too large), large objects (box too small), parts-only responses, context/look-alike evidence, and low classifier confidence, where the map is empty or off the object. Strengths: cheap, architecture-agnostic, class-specific. Limitations: coarse maps, sensitivity to layer, threshold and normalisation, and no guarantee that the highlighted region is causally used. Future work: deletion/insertion and occlusion faithfulness tests, a controlled normalisation ablation, segmentation-mask ground truth, other architectures (VGG-16, transformers), and Guided Grad-CAM.

References

[1] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[2] Aditya Chattopadhay, Anirban Sarkar, Prantik Howlader, and Vineeth N. Balasubramanian. Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 839–847, 2018.

[3] Rachel Lea Draelos and Lawrence Carin. Use HiResCAM instead of Grad-CAM for faithful explanations of convolutional neural networks. *arXiv preprint arXiv:2011.08891*, 2020.

[4] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979.

[5] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The PASCAL Visual Object Classes Challenge 2007 (VOC2007) results. http://www.pascal-network.org/challenges/VOC/voc2007/workshop/index.html, 2007.

[6] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The PASCAL visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[8] Joseph B. Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, 1964.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[10] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[11] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[12] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[13] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.

[14] Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.

[15] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[16] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[17] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014.

[18] Gurjeet Singh, Facundo Mémoli, and Gunnar Carlsson. Topological methods for the analysis of high dimensional data sets and 3D object recognition. In *Eurographics Symposium on Point-Based Graphics*, 2007.

[19] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, and Martin Riedmiller. Striving for simplicity: The all convolutional net. In *ICLR Workshop*, 2015.

[20] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

[21] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.

[22] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[23] Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European Conference on Computer Vision (ECCV)*, pages 818–833, 2014.

[24] Jianming Zhang, Sarah Adel Bargal, Zhe Lin, Jonathan Brandt, Xiaohui Shen, and Stan Sclaroff. Top-down neural attention by excitation backprop. *International Journal of Computer Vision*, 126(10):1084–1102, 2018.

[25] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. Learning deep features for discriminative localization. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2921–2929, 2016.